



Tsinghua University

Modern Computer Architecture Fall 2025

Lecture 08_1: Improving Cache Performance

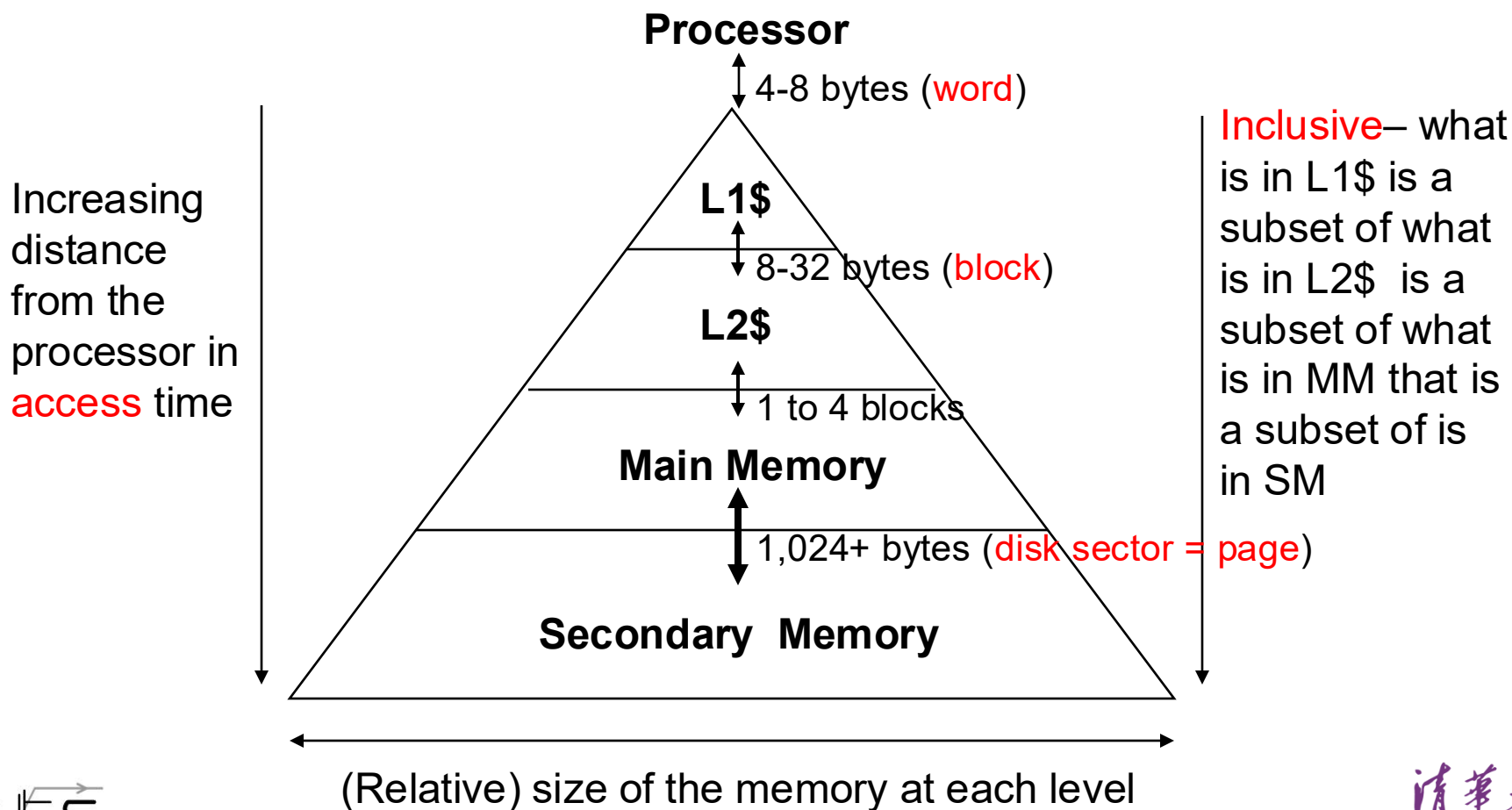
Yongpan Liu

ypliu@tsinghua.edu.cn

2025-Nov-19

Review: The Memory Hierarchy

- Take advantage of the principle of locality to present the user with as **much memory** as is available in the **cheapest** technology at the speed offered by the **fastest** technology



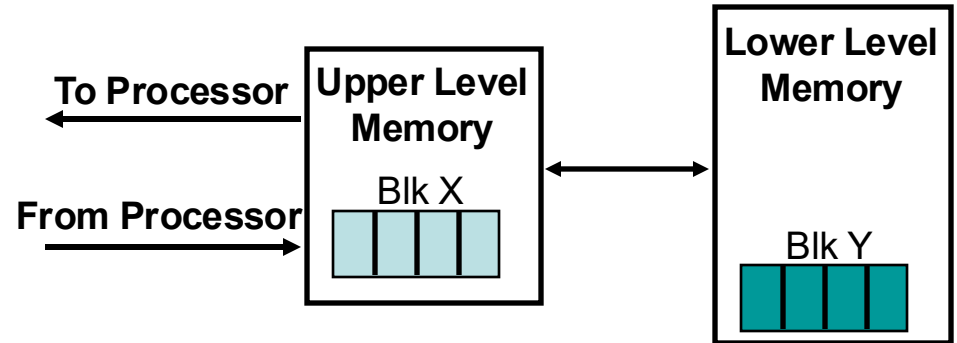
Review: Principle of Locality

- Temporal Locality

- Keep **most recently** accessed data items closer to the processor

- Spatial Locality

- Move **blocks** consisting of contiguous words to the upper levels



- Hit Time $\ll$ Miss Penalty

- **Hit**: data appears in some block in the upper level (Blk X)
 - **Hit Rate**: the fraction of accesses found in the upper level
 - **Hit Time**: RAM access time + Time to determine hit/miss
- **Miss**: data needs to be retrieve from a lower level block (Blk Y)
 - **Miss Rate** = $1 - (\text{Hit Rate})$
 - **Miss Penalty**: Time to replace a block in the upper level with a block from the lower level + Time to deliver this block's word to the processor
 - **Miss Types**: Compulsory, Conflict, Capacity

Measuring Cache Performance

- Assuming cache hit costs are included as part of the normal CPU execution cycle, then

$$\begin{aligned}\text{CPU time} &= IC \times CPI \times CCT \\ &= IC \times \underbrace{(CPI_{\text{ideal}} + \text{Memory-stall cycles})}_{CPI_{\text{stall}}} \times CCT\end{aligned}$$

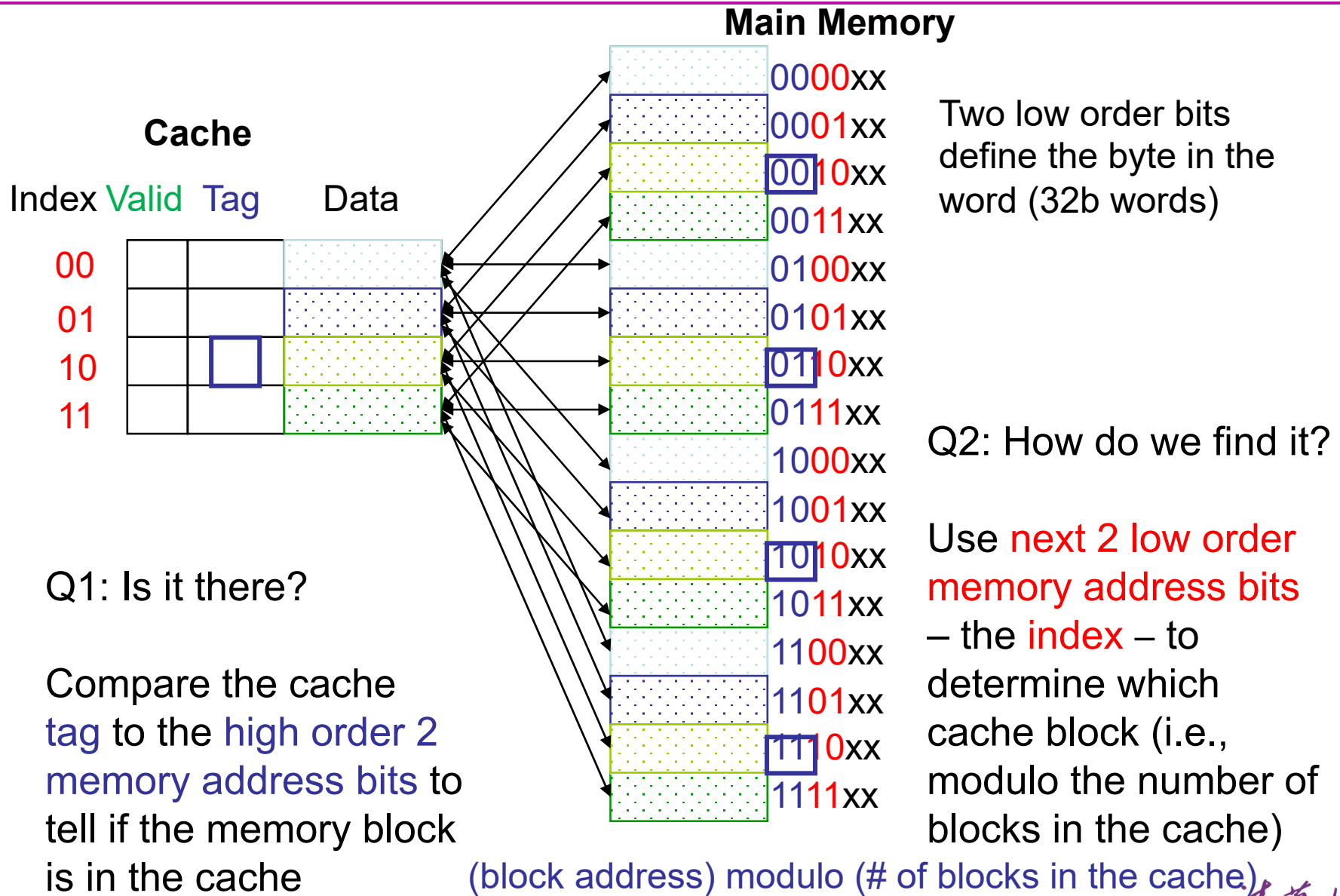
- Memory-stall cycles come from cache misses (a sum of read-stalls and write-stalls)

$$\begin{aligned}\text{Read-stall cycles} &= \text{reads/program} \times \text{read miss rate} \\ &\quad \times \text{read miss penalty}\end{aligned}$$

$$\begin{aligned}\text{Write-stall cycles} &= (\text{writes/program} \times \text{write miss rate} \\ &\quad \times \text{write miss penalty}) \\ &\quad + \text{write buffer stalls}\end{aligned}$$

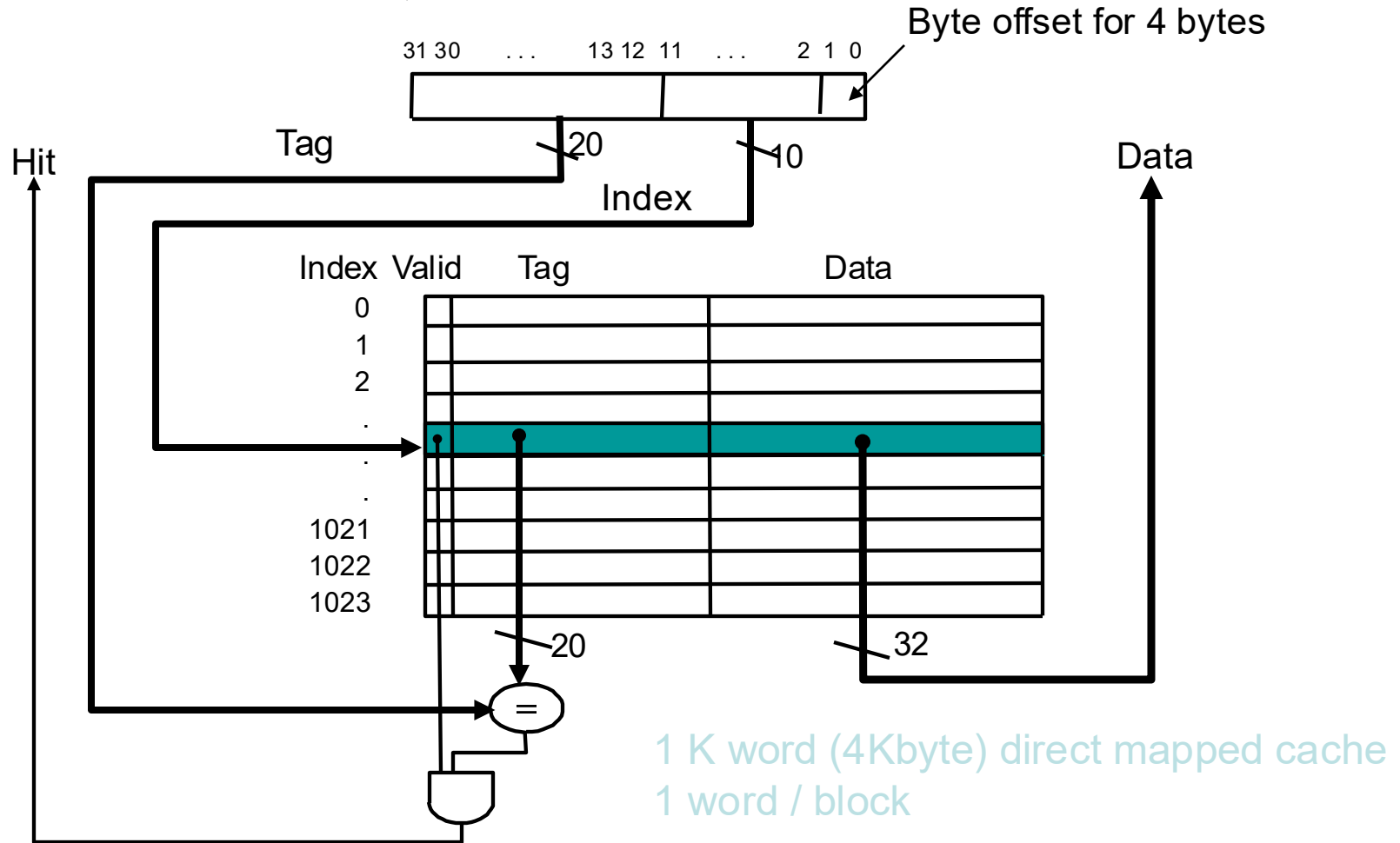
- For **write-through** caches, we can simplify this to
 - Memory-stall cycles = accesses/program $\times$ miss rate $\times$ miss penalty

Caching: A Simple First Example



Direct Mapped Cache Example

- One word/block, cache size = 1K words



What kind of locality are we taking advantage of?
Temporal!

Direct Mapped Cache

- Consider the main memory word reference string

Start with an empty cache - all blocks initially marked as not valid

0 1 2 3 4 3 4 15
0000 0100 1111

0 miss

00	Mem(0)

1 miss

00	Mem(0)
00	Mem(1)

2 miss

00	Mem(0)
00	Mem(1)
00	Mem(2)

3 miss

00	Mem(0)
00	Mem(1)
00	Mem(2)
00	Mem(3)

4 miss

01

00	Mem(0)
00	Mem(1)
00	Mem(2)
00	Mem(3)

4

3 hit

01	Mem(4)
00	Mem(1)
00	Mem(2)
00	Mem(3)

4 hit

01	Mem(4)
00	Mem(1)
00	Mem(2)
00	Mem(3)

15 miss

11

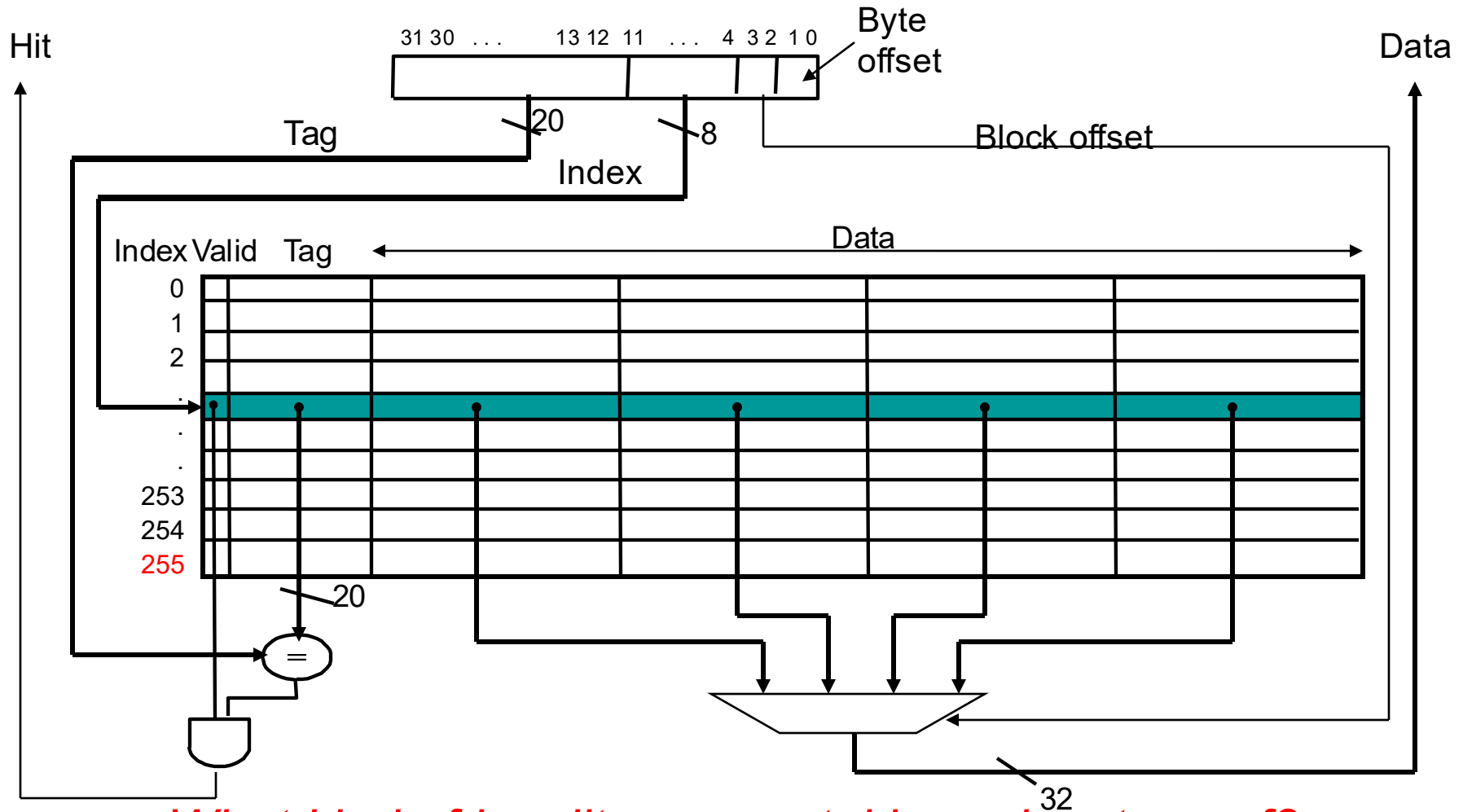
01	Mem(4)
00	Mem(1)
00	Mem(2)
00	Mem(3)

15

□ 8 requests, 6 misses

Multiword Block Direct Mapped Cache

- Four words/block, cache size = 1K words



What kind of locality are we taking advantage of?

Spatial ! + Temporal!

Taking Advantage of Spatial Locality

- Let cache block hold 2 words

Index	Tag	Data:1	Data:0	0	1	2	3	4	3	4	15	
					0000			0010			Tag(2): index(1): block offset(1)	
				0 miss			1 hit			2 miss		
0	00	Mem(1)	Mem(0)		00	Mem(1)	Mem(0)		00	Mem(1)	Mem(0)	
1									00	Mem(3)	Mem(2)	
				3 hit			4 miss			3 hit		
	00	Mem(1)	Mem(0)	01	Mem(1)	Mem(0)	01	Mem(5)	Mem(4)	01	Mem(5)	Mem(4)
	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)
				4 hit			15 miss					
	01	Mem(5)	Mem(4)	11	Mem(5)	Mem(4)	11	Mem(3)	Mem(2)	01	Mem(5)	Mem(4)
	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)

□ 8 requests, **4 misses**

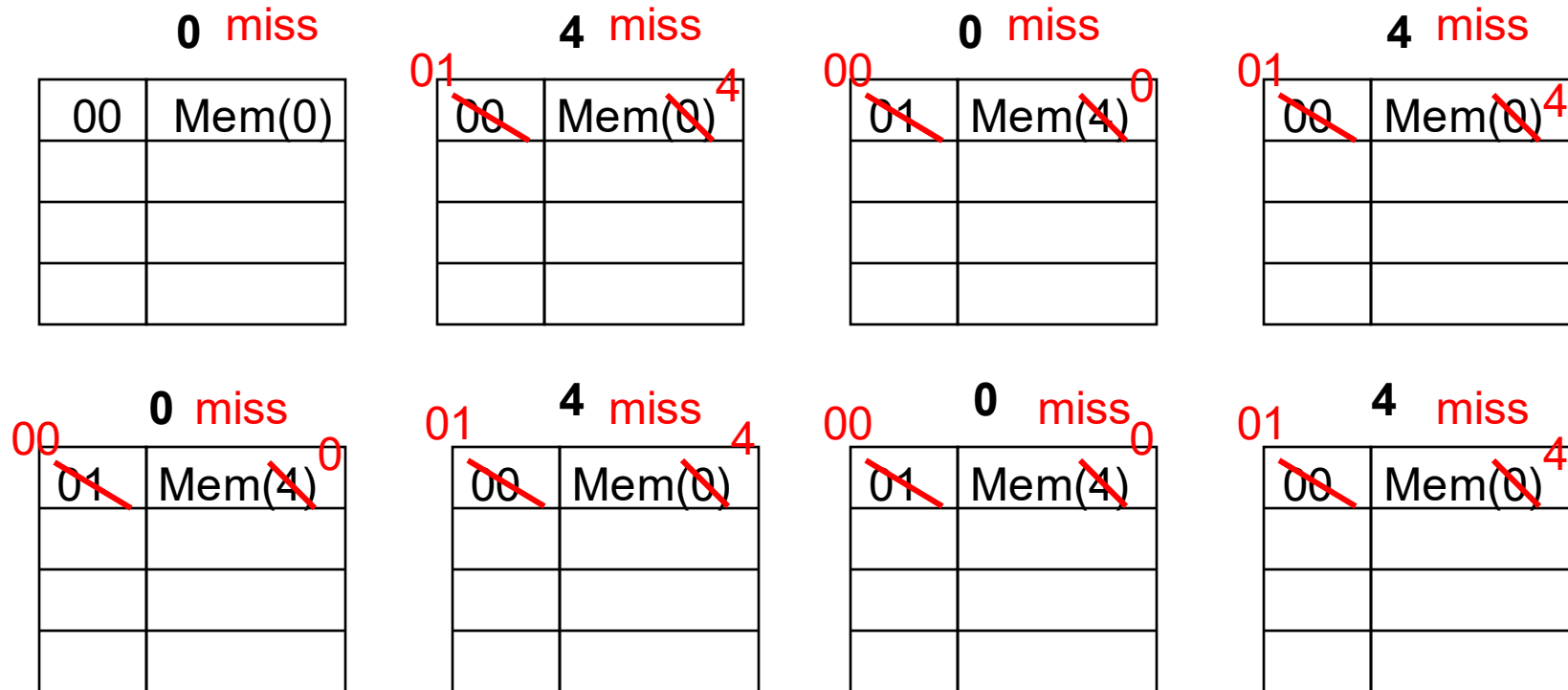
Start with an empty cache - all blocks initially marked as not valid

Another Reference String Mapping

- Consider the main memory word reference string

Start with an empty cache - all blocks initially marked as not valid

0 4 0 4 0 4 0 4



□ 8 requests, 8 misses

- Ping pong effect due to conflict misses - two memory locations that map into the same cache block

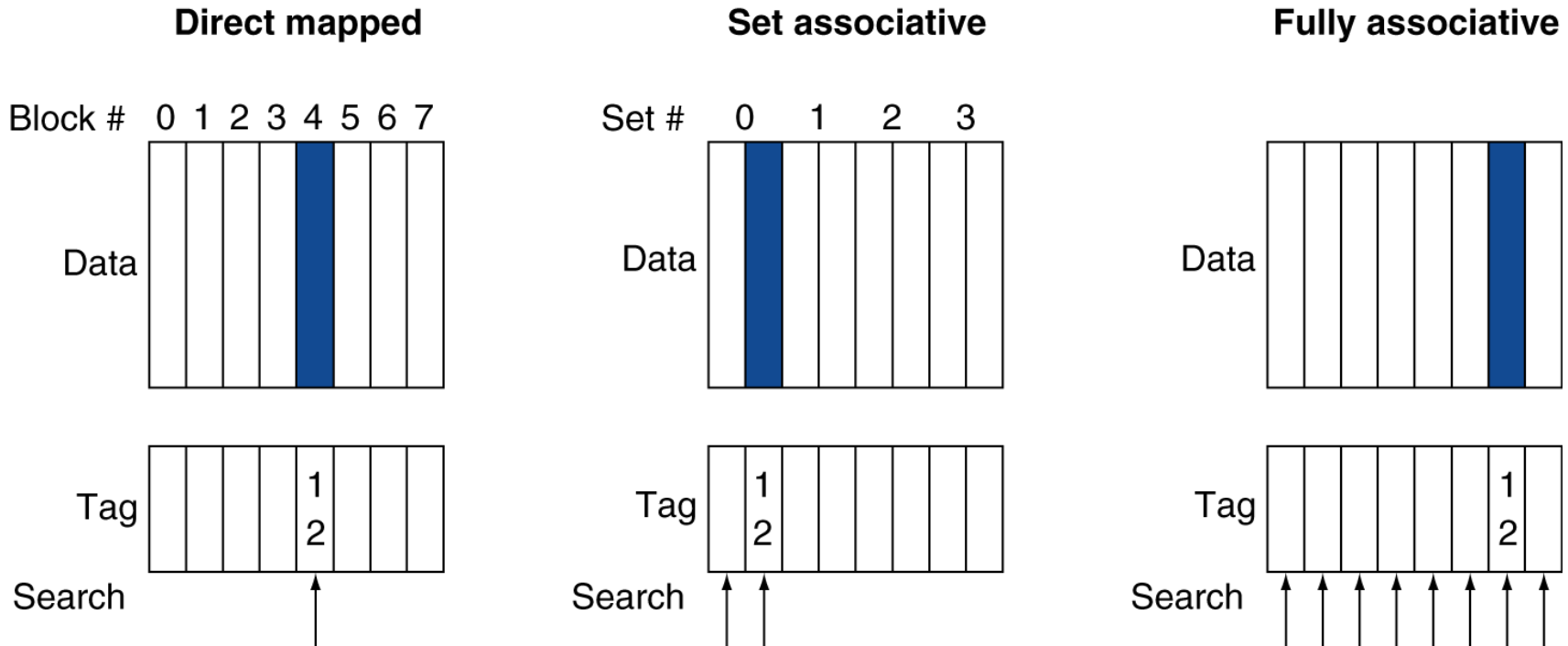
Cache Summary

- The Principle of Locality:
 - Program likely to access a relatively small portion of the address space at any instant of time
 - Temporal Locality: Locality in Time
 - Spatial Locality: Locality in Space
- Three major categories of cache misses:
 - Compulsory misses: sad facts of life. Example: cold start misses
 - Conflict misses: increase cache size and/or associativity
Nightmare Scenario: ping pong effect!
 - Capacity misses: increase cache size
- Cache design space
 - total size, block size, associativity (replacement policy)
 - write-hit policy (write-through, write-back)
 - write-miss policy (write allocate, write buffers)

Reducing Cache Miss Rates #1

1. Allow more flexible block placement
 - In a direct mapped cache a memory block maps to exactly one cache block
 - At the other extreme, could allow a memory block to be mapped to any cache block – fully associative cache
 - A compromise is to divide the cache into sets each of which consists of n “ways” (n-way set associative). A memory block maps to a unique set (specified by the index field) and can be placed in any way of that set (so there are n choices)
$$\text{set index} = (\text{MM block address}) \bmod (\# \text{ sets in the cache})$$

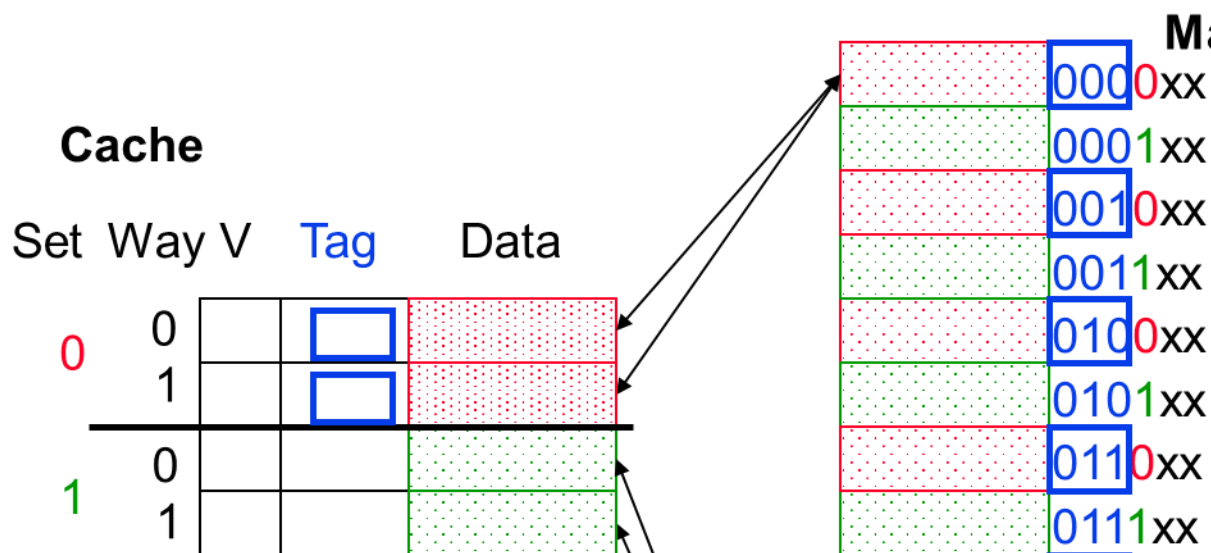
Cache Design Comparison



Direct mapped = 1-way Set associative
vs. 2-way Set associative
vs. Fully associative
each with total capacity of 8 blocks

Where do we find address **12**?
1-way set # $12 \bmod (8/1) = 4$
2-way set # $12 \bmod (8/2) = 0$

Set Associative Cache Example



One word blocks, so the two low order bits locate the byte in the word (32-bit words)

Q2: How do we find it?

Use **next 1 low order memory address bit** to determine which cache set (i.e., modulo the number of sets in the cache)

byte offset (2 bits)

set number (1 bit)

tag (3 bits)

Q1: Is it there?

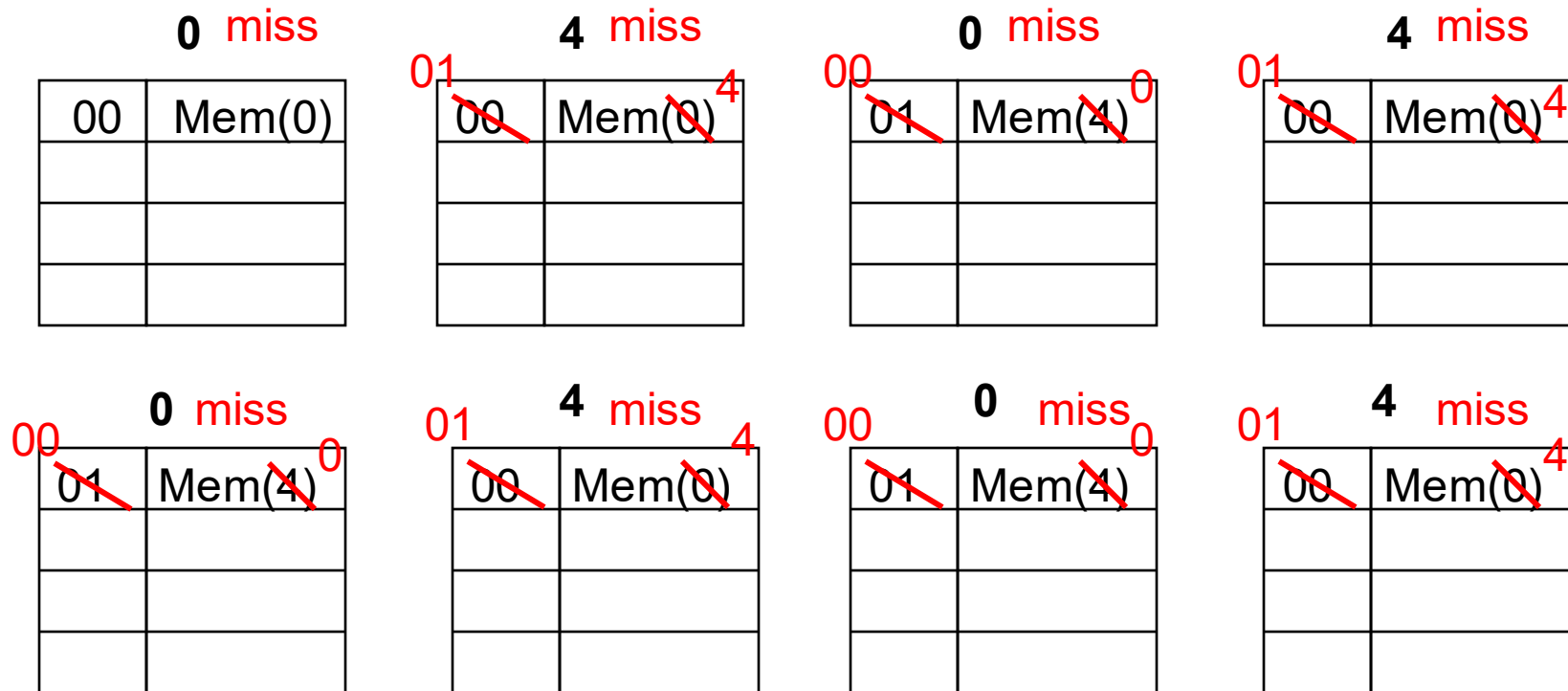
Compare *all* the cache **tags** in the set to the **3 high order memory address bits** to tell if the memory block is currently in the cache

Another Reference String Mapping

- Consider the main memory word reference string

Start with an empty cache - all
blocks initially marked as not valid

0 4 0 4 0 4 0 4



□ 8 requests, 8 misses

- Ping pong effect due to conflict misses - two memory locations that map into the same cache block

Another Reference String Mapping

- Consider the main memory word reference string

Start with an empty cache - all blocks initially marked as not valid

0 4 0 4 0 4 0 4

```
tag(3): set index(1)
```

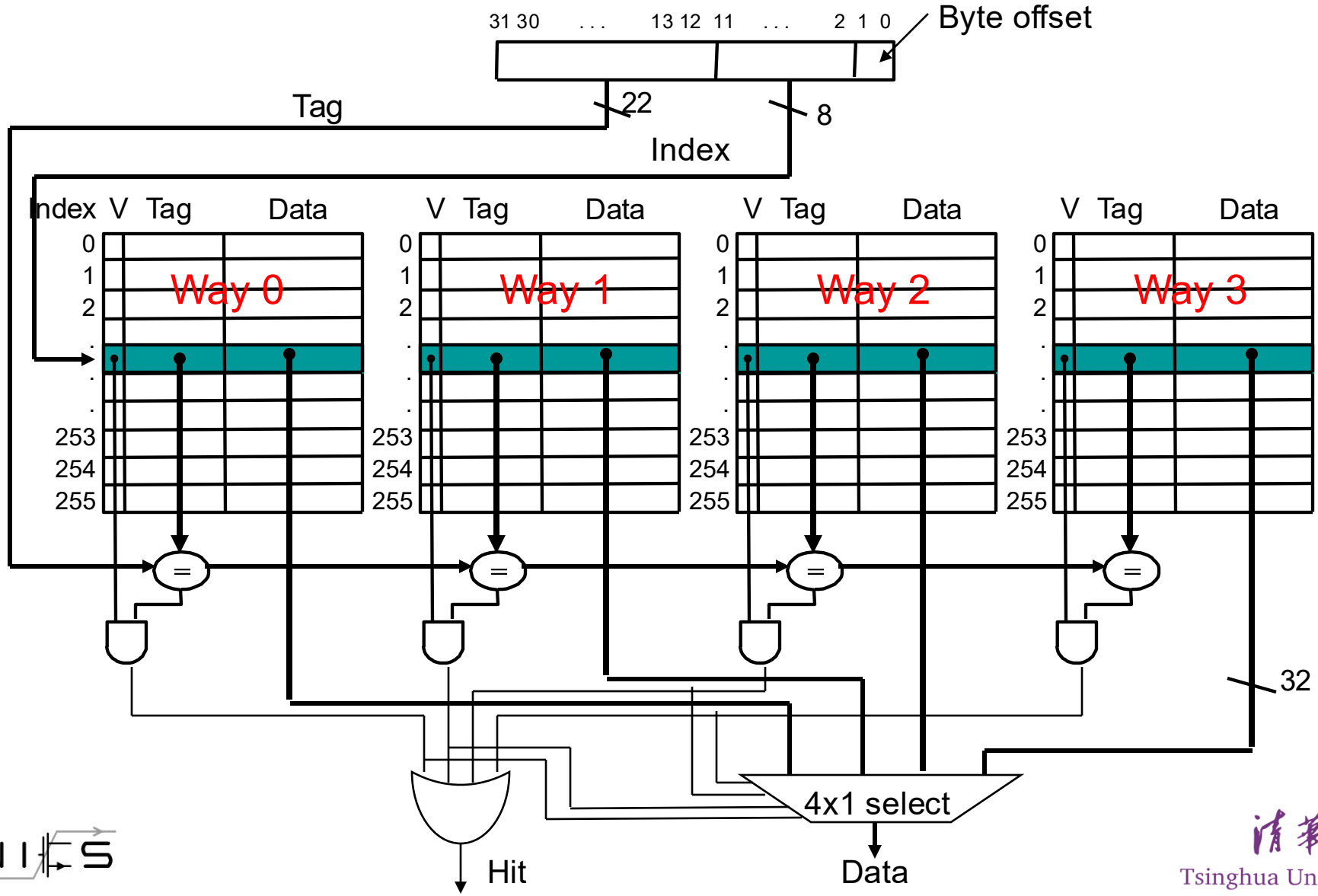
		0 miss		4 miss		0 hit		4 hit		
0	0	000	Mem(0)	000	Mem(0)	000	Mem(0)	000	Mem(0)	...
	1			010	Mem(4)	010	Mem(4)	010	Mem(4)	
1	0									
	1									

8 requests, 2 misses

- Solves the ping pong effect in a direct mapped cache due to **conflict** misses since now two memory locations that map into the same cache set can co-exist!

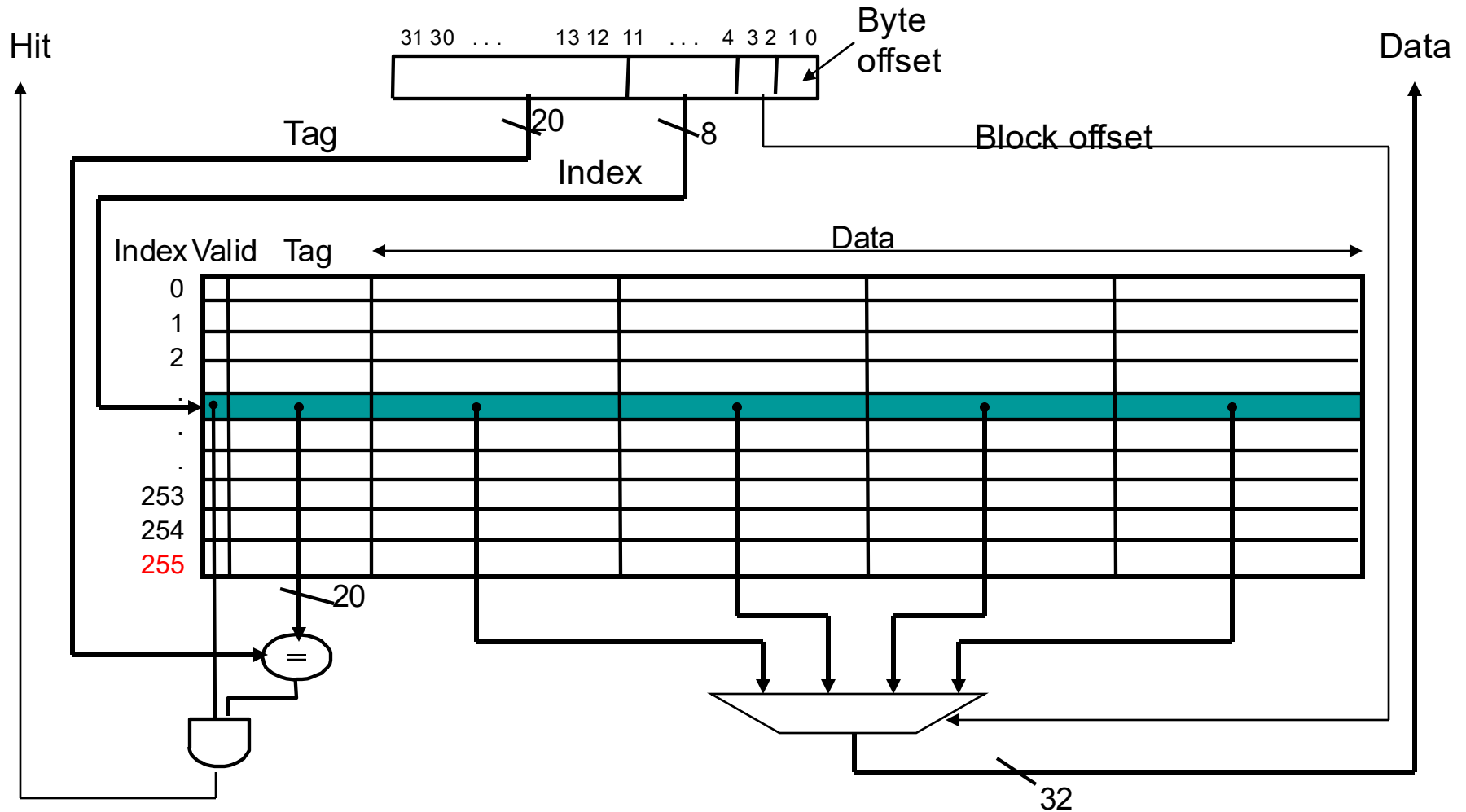
Four-Way Set Associative Cache

- $2^8 = 256$ sets each with four ways (each with one block)



Multiword Block Direct Mapped Cache

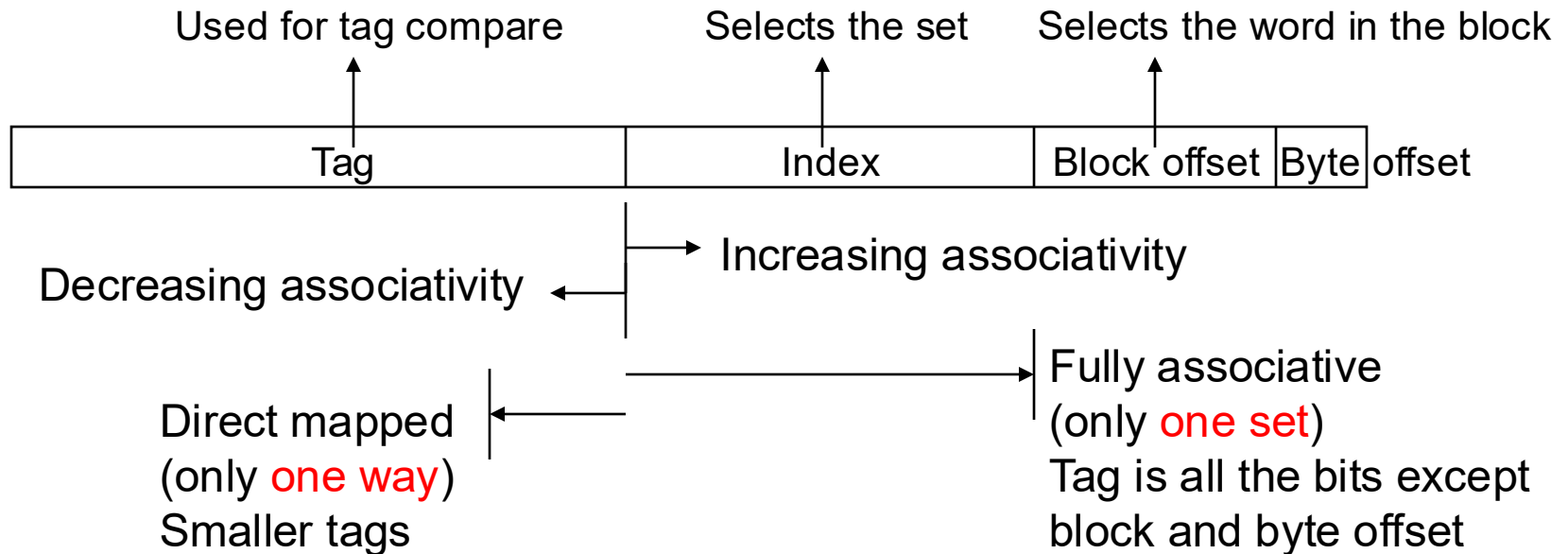
- Four words/block, cache size = 1K words



What is the Difference?

Range of Set Associative Caches

- For a fixed size cache, each increase by a factor of two in associativity **doubles the number of blocks** per set (i.e., the number of ways) and **halves the number of sets**
 - decreases the size of the index by 1 bit and increases the size of the tag by 1 bit

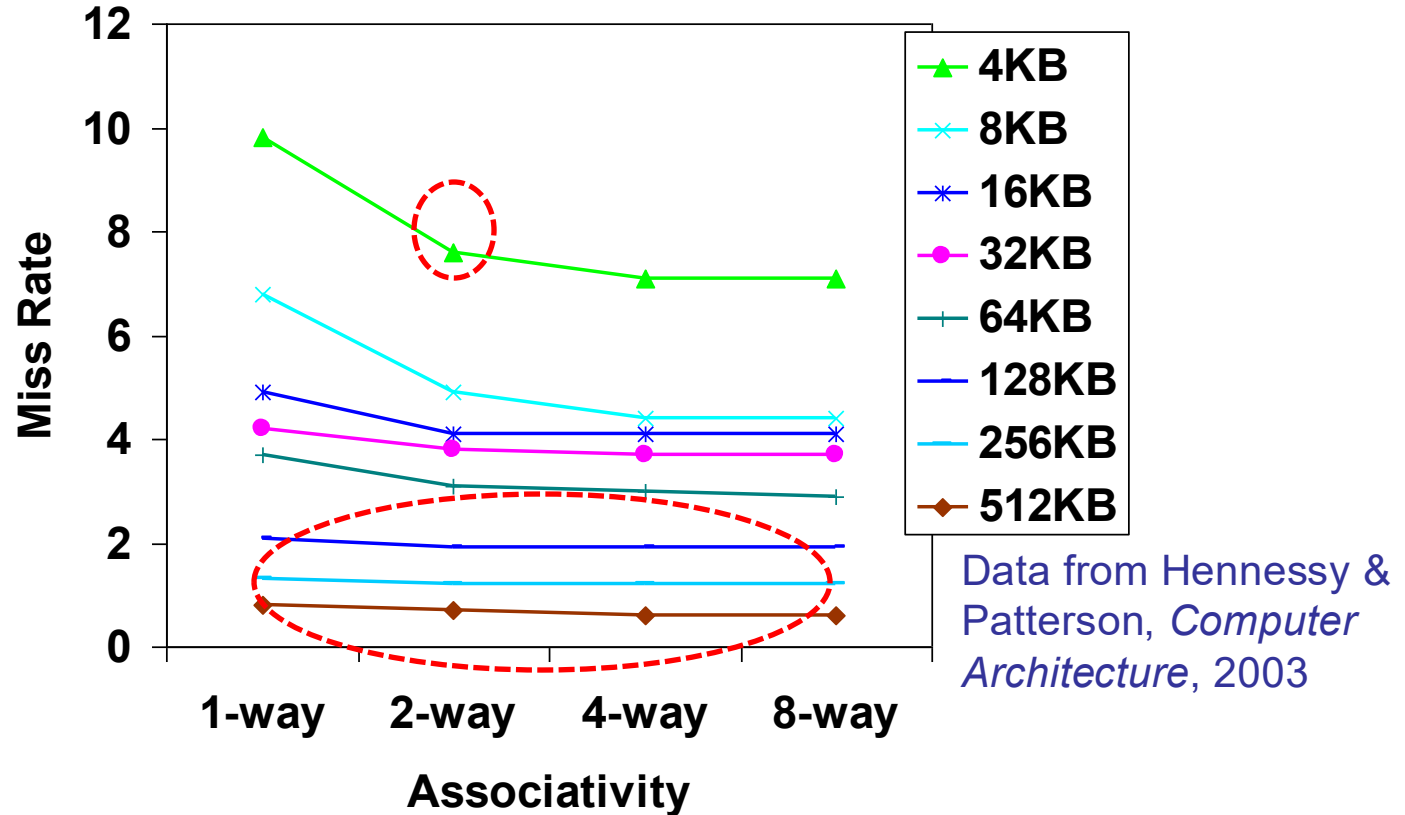


Costs of Set Associative Caches

- When a miss occurs, which way's block do we pick for replacement?
 - Least Recently Used (LRU): the block replaced is the one that has been unused for the longest time
 - Must have **hardware** to keep track of when each way's block was used relative to the other blocks in the set
- N-way set associative cache costs
 - N comparators (delay and area) for tag
 - MUX delay (set selection) before data is available

Benefits of Set Associative Caches

- The choice of direct mapped or set associative depends on the cost of a miss versus the cost of implementation



- Largest gains are in going from direct mapped to 2-way (20%+ reduction in miss rate)
- overall miss rate of a larger cache is lower

2. Use multiple levels of caches

– Also reduces cache miss penalty

- With advancing technology have more than enough room on the die for bigger L1 caches *or* for a second level of caches – normally a **unified** L2 cache (i.e., it holds both instructions and data) and in some cases even a unified L3 cache
- For our example, CPI_{ideal} of 2, 100 cycle miss penalty (to main memory), 36% load/stores, a **2% (4%)** L1I\$ (D\$) miss rate, add a UL2\$ that has a 25 cycle miss penalty and a **0.5%** miss rate

$$\begin{aligned}CPI_{stalls} &= 2 + .02 \times 25 + .36 \times .04 \times 25 && \text{L1-L2} \\ &\quad + .005 \times 100 + .36 \times .005 \times 100 && \text{L2-MM} \\ &= 3.54\end{aligned}$$

(as compared to 5.44 with no L2\$)

Multilevel Cache Design Considerations

- Design considerations for L1 and L2 caches are very different
 - Primary cache should focus on **minimizing hit time** in support of a shorter clock cycle
 - Smaller cache size with smaller block sizes
 - Secondary cache(s) should focus **on reducing miss rate** to reduce the penalty of long main memory access times
 - Larger cache size with larger block sizes
- The miss penalty of the L1 cache is significantly reduced by the presence of an L2 cache – so it can be smaller (i.e., faster) but have a higher miss rate

Key Cache Design Parameters

	L1 typical	L2 typical
Total size (blocks)	250 to 2000	4000 to 250,000
Total size (KB)	16 to 64	500 to 8000
Block size (B)	32 to 64	32 to 128
Miss penalty (clocks)	10 to 25	100 to 1000
Miss rates	2% to 5%	0.1% to 2% (global for L2)

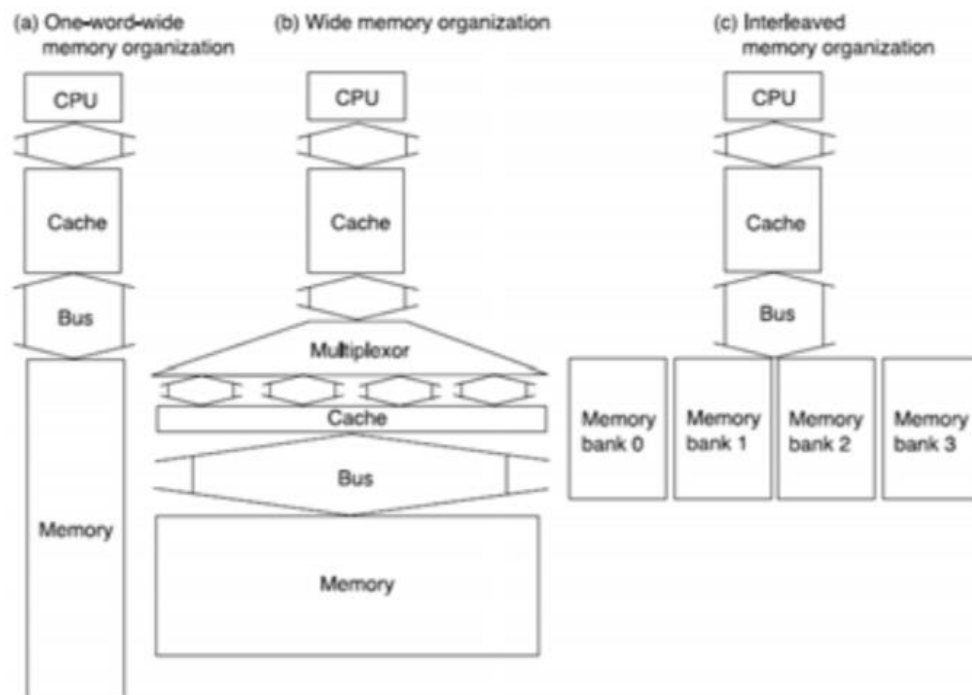
Two Machines' Cache Parameters

	Intel P4	AMD Opteron
L1 organization	Split I\$ and D\$	Split I\$ and D\$
L1 cache size	8KB for D\$, 96KB for I\$	64KB for each of I\$ and D\$
L1 block size	64 bytes	64 bytes
L1 associativity	4-way set assoc.	2-way set assoc.
L1 replacement	LRU	LRU
L1 write policy	write-through	write-back
L2 organization	Unified	Unified
L2 cache size	512KB	1024KB (1MB)
L2 block size	128 bytes	64 bytes
L2 associativity	8-way set assoc.	16-way set assoc.
L2 replacement	LRU	LRU
L2 write policy	write-back	write-back

Improving Cache Performance

- Assumptions

- cache block of 4 words
 - $4 \times 4 = 16$ bytes of data
- 1 clock cycle to send block address on bus
 - sufficient to send it once
- 15 clock cycles to initiate each DRAM access
- 1 clock cycle to transfer each data word



- Miss penalty P and memory bandwidth B per organization

a) $P = 1 + 4 \times 15 + 4 \times 1 = 65$

($B = 16/65 = 0.25_{\text{bytes/cycle}}$)

b) {w/ 2 words} $P = 1 + 2 \times 15 + 2 \times 1 = 33$

($B = 16/33 = 0.48_{\text{bytes/cycle}}$)

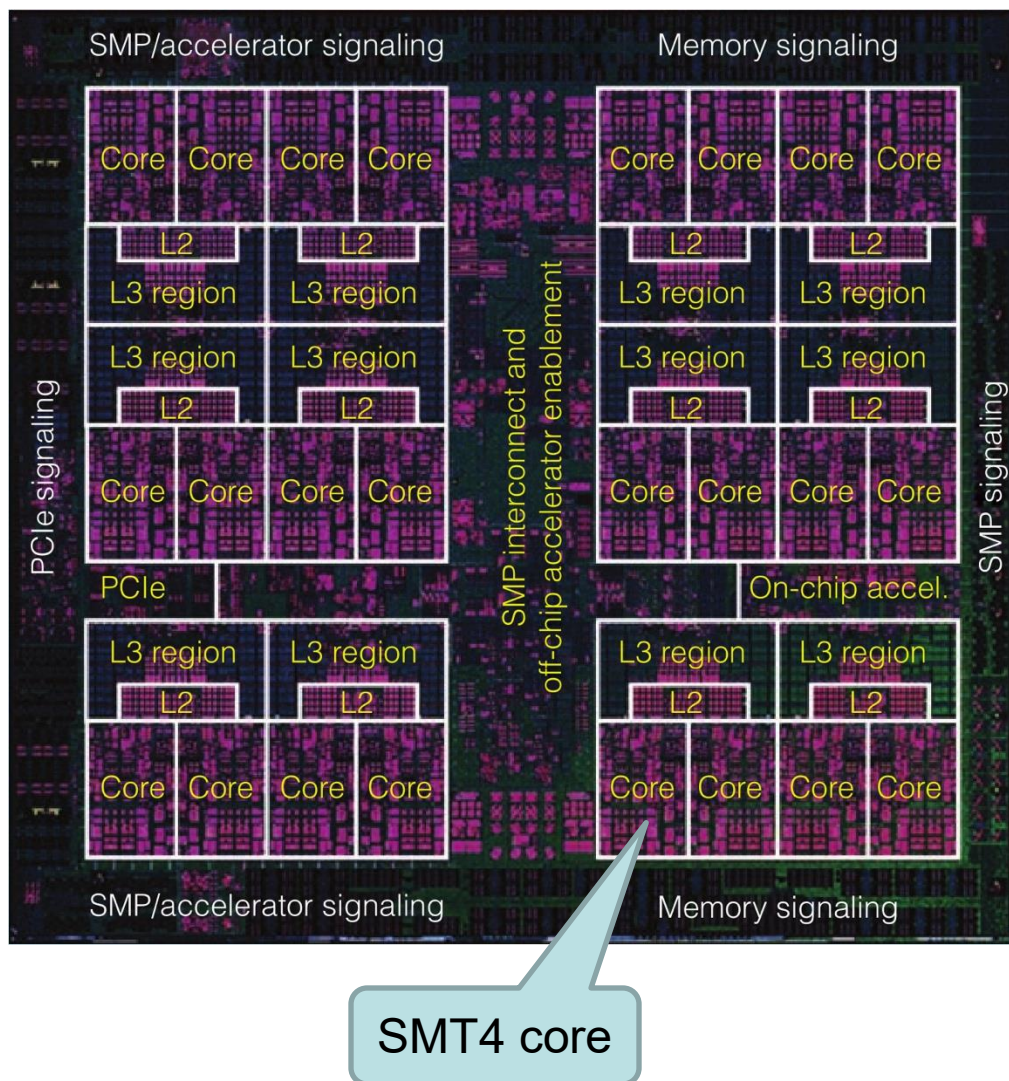
b) {w/ 4 words} $P = 1 + 1 \times 15 + 1 \times 1 = 17$

($B = 16/17 = 0.94_{\text{bytes/cycle}}$)

c) $P = 1 + 1 \times 15 + 4 \times 1 = 20$

($B = 16/20 = 0.80_{\text{bytes/cycle}}$)

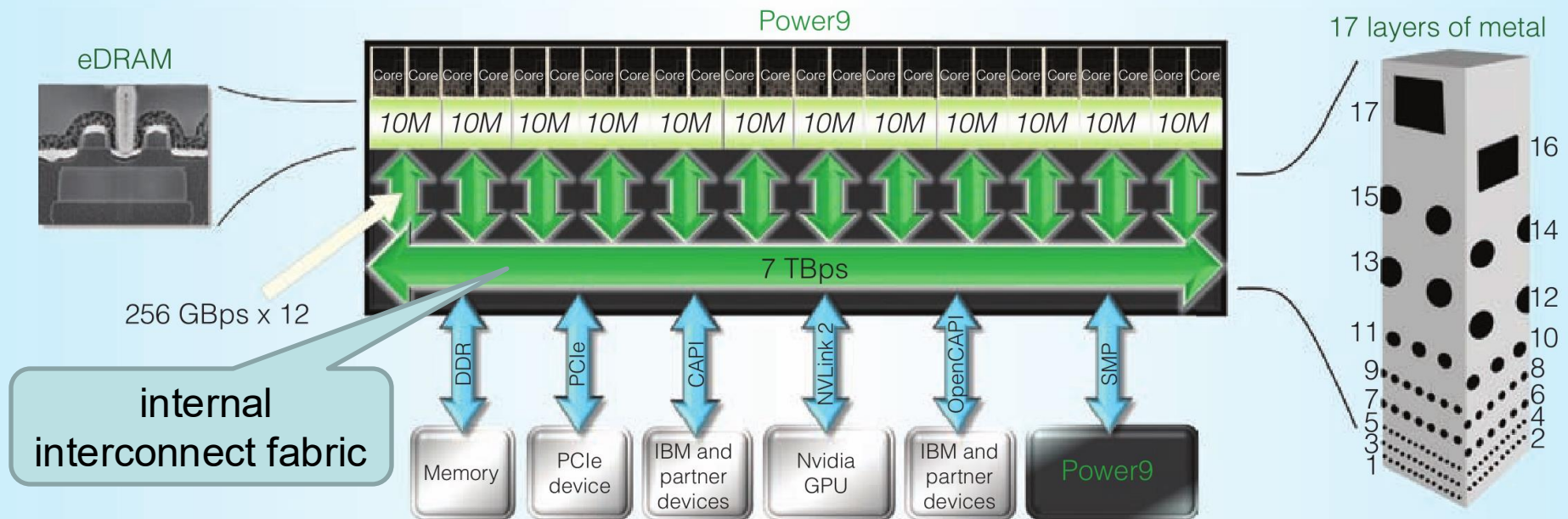
IBM Power9 Cache&Memory System



- Each SMT4 core has a private 32-KB eight-way L1 cache
- L2 and L3 cache regions are shared by 2 SMT4 cores :
 - 512-KB, eight-way set-associative L2 cache
 - 10-MB, 20-way set-associative L3 cache

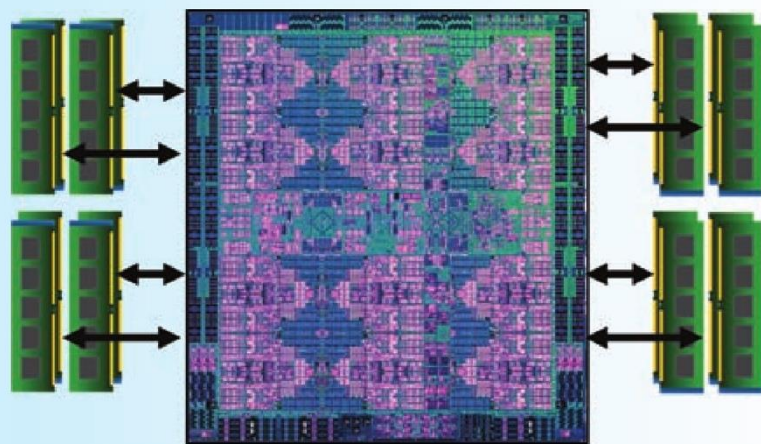
IBM Power9 L3 Cache Architecture

- A total of 120-MB of eDRAM-based L3 cache
- 12 regions of 10 MB each 2 SMT4 cores
 - Each region shared by up to eight threads
- Each L3 region can be accessed by other cores
 - High-throughput on-chip interconnect fabric
 - provide an aggregate 7.3 TB/s for on-chip data switching
 - Optimized for data-intensive workloads



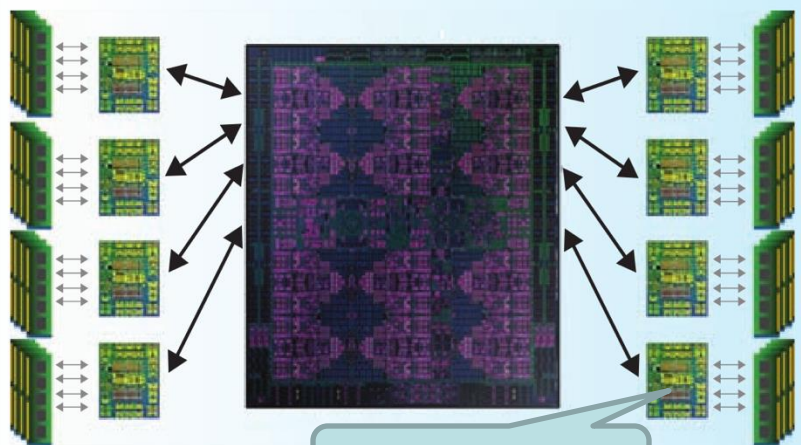
IBM Power9 Cache&Memory System

- Power9 memory system supports scale-out & scale-up domains



(a) Scale-out

One socket

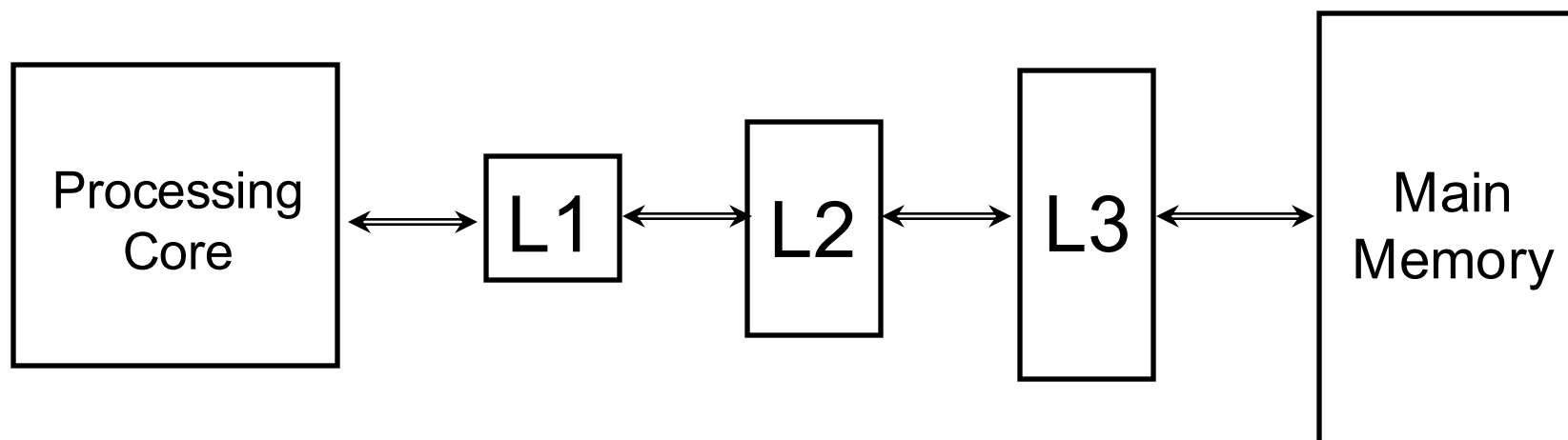


(b) Scale-up

buffered memory

- Toward scale-out domain :
 - Directly attached to DDR4
 - Supporting up to 120 GB/s of sustained memory bandwidth with a maximum memory capacity of up to 4 TB per socket
- Toward scale-up domain
 - 8 buffered memory are introduced
 - Buffered memory then attached to DDR4
 - Supporting a sustained bandwidth of 230 GB/s, with a maximum memory capacity of up to 8 TB per socket.

多级cache降低miss penalty



访问时间 $t_{L1}=1 \text{ clk}$ $t_{L2}=2 \text{ clks}$ $t_{L3}=5 \text{ clks}$ $t_{MM}=20 \text{ clks}$

假定命中率 $h_{L1} = 70\%$, $h_{L2} = 80\%$, $h_{L3} = 90\%$

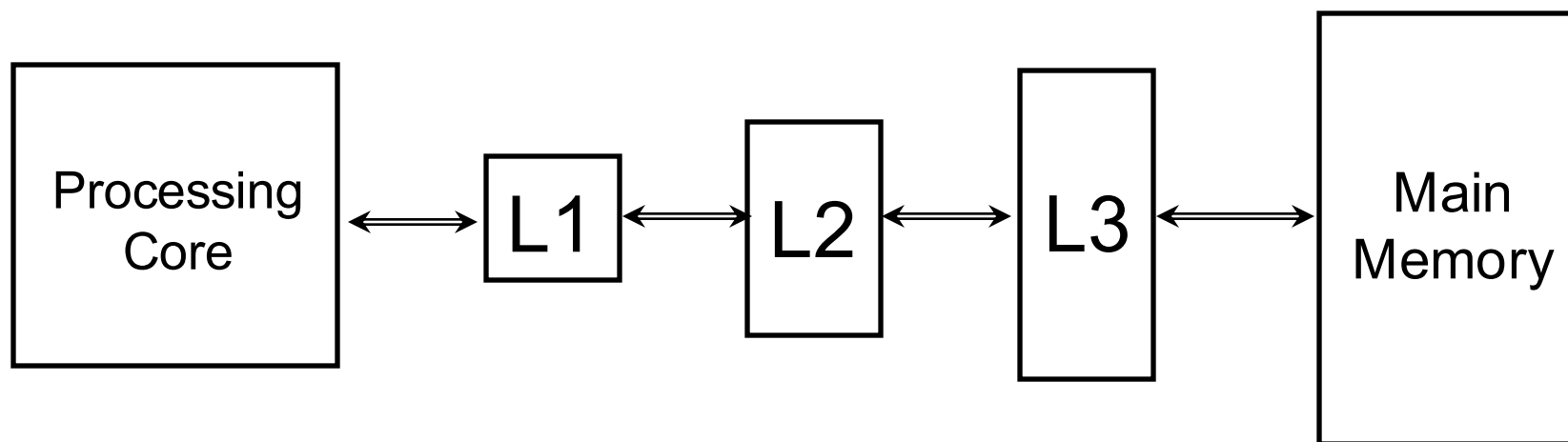
$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

$$\text{平均访问时间(AMAT)} = t_{L1} + (1-h_{L1})[t_{L2} + (1-h_{L2})(t_{L3} + (1-h_{L3})t_{MM})]$$

=2.02clks

如果仅有L1&L2, AMAT=2.8clks

多级cache降低miss penalty



访问时间	$t_{L1} = 1 \text{ clk}$	$t_{L2} = 2 \text{ clks}$	$t_{L3} = 5 \text{ clks}$	$t_{MM} = 20 \text{ clks}$
假定命中率	$h_{L1} = 70\%$	$h_{L2} = 80\%$	$h_{L3} = 90\%$	

$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

- L2, L3 Cache的引入降低了对主存的直接访问，使得L1 Cache的miss penalty大大降低，故L1 Cache通常较小以获得更快的访问速度（命中时间）
- 由于L1 Cache的存在，L2, L3 Cache的访问时间仅影响L1 Cache的缺失损失，不直接影响L1 Cache的访问时间，故L2, L3 Cache通常容量较大，以获得更高的命中率

4 Questions for the Memory Hierarchy

- Q1: Where can a block be placed in the upper level?
(Block placement)
- Q2: How is a block found if it is in the upper level?
(Block identification)
- Q3: Which block should be replaced on a miss?
(Block replacement)
- Q4: What happens on a write?
(Write strategy)

Q1&Q2: Where can a block be placed/found?

	# of sets	Blocks per set
Direct mapped	# of blocks in cache	1
Set associative	(# of blocks in cache)/ associativity	Associativity (typically 2 to 16)
Fully associative	1	# of blocks in cache

	Location method	# of comparisons
Direct mapped	Index	1
Set associative	Index the set; compare set's tags	Degree of associativity
Fully associative	Compare all blocks tags	# of blocks

Q3: Which block should be replaced on a miss?

- Easy for direct mapped – only one choice
- Set associative or fully associative
 - Random
 - LRU (Least Recently Used) – hardware supporting
- For a 2-way set associative cache, random replacement has a miss rate about 1.1 times higher than LRU (typical).
- LRU is too costly to implement for high levels of associativity (> 4 -way) since tracking the usage information is costly

Q4: What happens on a write?

- Write-through – The information is written to both the block in the cache and to the block in the next lower level of the memory hierarchy
 - Write-through is always combined with a write buffer, so write waits to lower level memory can be eliminated (as long as the write buffer doesn't fill)
- Write-back – The information is written only to the block in the cache. The modified cache block is written to main memory only when it is replaced.
 - Need a dirty bit to keep track of whether the block is clean or dirty

Improving Cache Performance

$$AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

1. Reduce the time to hit in the cache

- smaller cache
- direct mapped cache
- smaller blocks

2. Reduce the miss rate

- bigger cache
- more flexible placement (increase associativity)
- larger blocks (16 to 64 bytes typical)

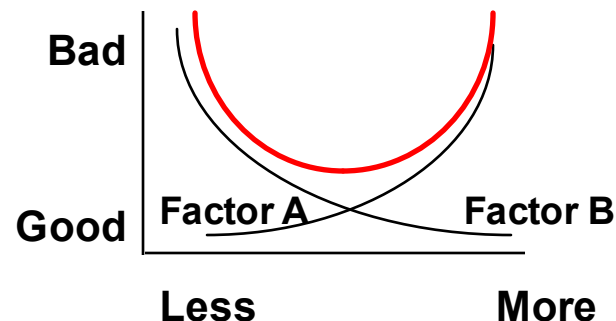
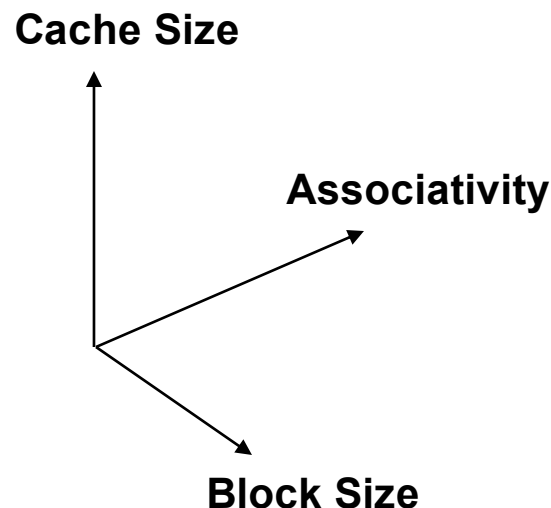
Improving Cache Performance

3. Reduce the miss penalty

- smaller blocks
- check write buffer on read miss
- for large blocks fetch critical word first
- use multiple cache levels – L2 / L3
- faster backing store/improved memory bandwidth
 - wider buses
 - memory interleaving, page mode DRAMs

Summary: The Cache Design Space

- Several interacting dimensions
 - cache size
 - block size
 - associativity
 - replacement policy
 - write-through vs write-back
 - write allocation
- The optimal choice is a compromise
 - depends on access characteristics
 - workload
 - use (I-cache, D-cache, TLB)
 - depends on technology / cost



NIS **Simplicity often wins (e.g. LRU)**